Project Master Document: Unusual Electricity Use in University Buildings

Oct 28, 2026 · @barnn

The project in plain English

Large buildings use a lot of electricity, and nobody can watch every meter every hour. We are building a tool that points a person to the few hours worth a second look. It does not say that something is wrong. It says that the use looks unusual for that building at that time.

How the tool works. Think of a caretaker who knows a building's routine: busy on weekday mornings, quiet at night and on weekends, more cooling on hot days. For every hour, the tool works out how much electricity the building should be using, based on the time, the weather and facts about the building, and on what it used just before. Then it compares that with what was really used. A small gap is normal. A big gap, judged against that building's own usual gaps, gets marked.

How we know whether it is any good. We use a public data set in which people looked at graphs of building electricity use and marked the stretches that looked unusual. We hide these marks from the tool and use them afterwards as an answer key. We ask two things: of the stretches people marked, how many did the tool also mark, and of the places the tool marked, how many matched a stretch people marked.

What we have achieved so far.

- We cleaned the data, for example by treating long runs of zero readings as a broken meter and not as a quiet building.
- We set up a fair test. The tool learns from the earlier months and is tested on the latest months, which it has never seen. The line between normal and unusual is decided before we look at the test months.
- We built six ways of marking unusual hours, from very simple ones to a learning model, and compared them under the same rules.
- We found that the use in the hour just before is the most helpful clue, and that a very simple rule, which assumes this hour will be like the last one, does almost as well as the learning model. The learning model does better on problems that last many hours. The simple rule does better on short ones.
- We found that the tool catches some of the stretches people marked, but marks many hours that nobody marked. So it is a helpful way to rank hours for a person to check, and not a finished alarm system.

- We were open about one problem: our first cut-offs marked very different numbers of hours across methods. We noticed it after looking at the test results, so we kept the cut-offs, reported it, and added a second comparison that gives every method the same number of marks.
- We wrote the midterm report, the slides and one notebook that runs the whole thing from start to finish.

What is still to do (items above marked Done are finished; the rest is below).

- **Done on 2 November: we tested whether using the people's marks from the earlier months to draw a better line for each building helps. We kept the single shared line, as the 2 November update explain**s.
- **Done on 2 November: we added a different kind of detector (an Isolation Forest) that looks directly for hours unlike all the others, and compared it with the other methods. The 2 November update gives the result**.
- **Done on 2 November (notebook 09): we tested the professor's question about long problems directly. If a fault keeps use high for days, does a tool that learns from the recent past get used to it and stop marking? The persistence section of the 2 November update gives the answer. A second view on CU-BEMS is still to do**.
- **Done on 2 November (notebooks 08 and 10): we checked how many hours inside each stretch the tool marks and how late it starts, gave honest margins of error (resampling whole buildings), and looked at 30 marks that matched nothing. Claude rated those 30 marks, so the team still has to cross-check the ratings**.
- **Done on 9 November: tested on a second data set (see the update of 9 November).** It comes from one office building in Bangkok. It has no marks by people, so we plant fake problems at times we choose and see whether the tool finds them. This tells us whether our findings hold elsewhere. Planted problems are easier than real ones, so we will not present it as proof that the tool works in real life.
- **Produce the final result:** a ranked list of unusual hours for each building that a person can check, a final report and presentation, and code that anyone can run again.

What the project will not claim. That a marked hour is waste, a fault or a saving. Public data tells us nothing about who was in the building or what was repaired. We claim only that the hour was unusual for that building at that time.

Status on 2 November 2026: the midterm part is finished and presented, and the follow-up work up to task T29 in the Complete Task List is done (full clean re-run, Isolation Forest, equal alarm rates with margins of error, label-guided cut-off check, retraining without labeled hours, the persistent-anomaly test and the review of flags that match no label). It built and tested a way to flag unusual hours of electricity use and checked the flags against labels made by people. What is left: the second dataset (CU-BEMS) with planted problems, which waits until it is downloaded, a few extras (a ranked review list, a

filter for site-wide events, which features matter), and the final deliverables (tables and figures, report, slides, script, tidy-up). Read the 2 November update first, because it is newer than the older sections. This document is the single reference for the next steps. Every statement about what we built was checked against the midterm notebook, and every statement about CU-BEMS was checked against its data paper.

The research question. How well do residual-based flags from an expected-consumption model match labeled anomalies in building electricity data, and which design choices (features, threshold, detector) improve that match? The professor also asked whether better prediction from historical consumption makes the model overlook persistent anomalies.

What an anomaly means here. Use that is unusual given its context. It is not proof of waste, a fault or a saving. The output is a ranked list for a person to review.

Start here: a walkthrough for someone new to the project (updated 2 November 2026)

The team and the course. Barnnita Shrestha and Arpan Ojha, graduate students at the Asian Institute of Technology (AIT), course AT82.01 Computer Programming for Data Science and AI, instructor Dr. Chantri Polprasert. Final deadline: 17 November 2026 (a report in the AIT thesis template and a presentation). The report needs: Introduction, Problem statement, Related works, Datasets (at least three), Methodology (exploratory analysis, pre-processing, ML model, pipeline), Model evaluation results, Discussions (performance, interpretability, complexity) and Conclusions, with sources cited and a baseline described.

The question. How well do flags raised from the gap between actual and expected electricity use match the stretches that people marked as unusual in building data, and which design choices (features, cut-off, detector) improve that match? The professor added a question: if the model predicts better from historical use, does it start to overlook anomalies that last a long time, because it learns them as normal?

The idea in five steps. (1) Clean the readings. (2) Predict what each building should use in each hour. (3) Score each hour by how far the actual use is from the prediction, measured against that building's own usual gaps. (4) Flag the hours whose score passes a cut-off that was fixed in advance using only normal validation hours. (5) Check the flags against the human labels. An anomaly here means use that is unusual for its context. It is not proof of waste, a fault or a saving. The output is a ranked list for a person to review.

The three datasets. (1) GEPIII, the ASHRAE Great Energy Predictor III competition data from Kaggle: hourly meter readings for 2016, weather, and building facts. We use meter 0, which is electricity, for the 200 buildings that have labels, about 1.75 million hourly rows.

GEPIII's buildings come from the Building Data Genome Project 2 (BDG2), so BDG2 is the source of GEPIII and not a separate dataset. (2) LEAD 1.0: human-made labels of unusual hours for GEPIII buildings (37,296 labeled hours that form 5,811 events). The labels check our flags against human judgment but are not independent of the readings. (3) CU-BEMS: one office building in Bangkok with no labels and no weather file, so we plant anomalies of known size and position and check whether the pipeline finds them. CU-BEMS is not used yet.

Words you will meet.

- *Hour / reading:* one electricity value (kWh) for one building and one hour.
- *Expected use / prediction:* what a method says the hour should have been.
- *Error:* the difference between actual and expected, measured on log(1 + kWh) so that a 10 kWh gap matters in a small building and not in a huge one.
- *Score:* the error, shifted by the building's usual error and divided by the building's usual spread (median and scaled median absolute deviation).
- *Cut-off:* the score above which an hour is flagged. One per method, fixed from normal validation hours before the test is scored.
- *Flag:* an hour whose score is above the cut-off.
- *Event:* a stretch of labeled hours in LEAD. An event is *found* when at least one flagged hour falls inside it.
- *Recall (events found):* the share of events that are found.
- *Run precision:* the share of runs of consecutive flagged hours that touch a labeled hour.
- *Equal alarm rate:* every method flags the same share of hours (0.5%, 1%, 2%), so no method wins just by flagging more.
- *Pooled / per building:* one shared threshold for all buildings, or each building flags its own top hours.
- *Baseline:* a simple method to beat. Ours are same-as-last-hour, historical typical value and a fixed percentile.
- *Model A, B, C:* the same gradient-boosting model with more and more history: A has none, B adds the same hour yesterday and last week, C also adds the previous hour.
- *Persistent anomaly:* one that lasts many hours; the risk is that a model using the previous hour learns it as normal after the first hour.
- *Explain-away filter:* a step that marks a flag as explained when most buildings on the same site deviate at the same time (a site-wide event).

The six methods (and two Isolation Forest variants). Same as last hour; historical typical value (median of the building's training use by hour and weekend, falling back to hour only, then to the building median); Model A; Model B; Model C; fixed percentile (flag use above

the building's own 99.5th percentile of clean training use); Isolation Forest on Model C error; Isolation Forest on raw readings. Models A, B and C are scikit-learn's HistGradientBoostingRegressor on log(1 + kWh): 600 rounds, learning rate 0.05, at most 127 leaves, at least 100 rows per leaf, L2 penalty 1.0, seed 0, no tuning, no early stopping. Model A uses 15 features (hour, day of week, weekend flag, building identity, site, primary use, floor area, year built, floor count, and the six weather variables air temperature, dew point, cloud cover, rain in the last hour, sea-level pressure and wind speed). Model B adds lag_24 and lag_168 (17 features). Model C adds lag_1 (18 features). Weather is filled forward in time by site and never backward. The month is not used because the test months do not appear in training. Wind direction is not used.

The split. One cut in time for everyone. Train: before 13 September 2016. Validation: 13 September to 6 November. Test: 7 November to 31 December. We test on a later period, not on unseen buildings, because the building identity is a feature.

Cleaning rules. A run of 24 or more exact zeros counts as missing, not as real low use. Site 0 is in kBTU and is converted to kWh by multiplying by 0.2931. Every building gets a complete grid of 8,784 hours; missing hours stay empty. Lags are looked up by timestamp, not by row, because about 0.074% of rows are not exactly one hour apart.

The folder (Project/). ashrae-energy-prediction/ holds the raw files (train.csv, weather_train.csv, building_metadata.csv, train_features.csv). notebooks/ holds config.py (all shared settings) and notebooks 01 to 08 (see below). outputs/ holds the 29 frozen midterm files, never edited. outputs_split/ is where notebooks write their results. results/ holds small copies of final result tables. docs/ holds decision.md, project_explained.md and ACTIONPOINTS.md. reports/ is for the report and slides. archive/ holds older work. Electricity_Anomaly_Detection_Midterm.ipynb is the combined midterm notebook.

The notebooks, in run order. 01 builds LEAD labels, matches them to GEPIII and makes events by length. 02 checks the GEPIII data. 03 builds features, the simple predictors and Models A, B, C (the slow one, about 5 minutes on a fast machine). 04 computes scores, cut-offs and flags and saves the cut-offs before the test is scored. 05 evaluates flags against events. 06 does the equal-alarm-rate comparison. 07 checks that the results agree with the midterm. 08 adds the Isolation Forest, the margins of error and the cut-off rule check. 09 is the persistent-anomaly test (recall by event length and the flag rate by position inside an event). 10 reviews flagged runs that match no label and writes the review pack. A separate notebook in notebooks/label_excluded/ retrains the models without the labeled hours. requirements.txt in the project folder lists the Python packages (Python 3.13, numpy 2.5.3, pandas 3.0.5, scikit-learn 1.9.1, matplotlib 3.11.2). Every notebook starts with from config import *. To run everything: open each in order and run all cells. If notebook 07 says DIFFERENT, or the ranking changes, stop and look before going on.

Decisions already made (docs/decision.md). Deadline 17 November 2026. No comments from the professor after the midterm. Test protection: the test period is scored once with frozen settings (option A, assumed, to be confirmed by the team). CU-BEMS stays as the third dataset. BDG2 is described as the source of GEPIII. Holiday limitation: the data has no holiday calendar and no site locations, so we state this as a limit and use only the simple site-wide explain-away filter.

What we will not claim. That a flagged hour is waste, a fault or a saving. That the system beats every method: at equal alarm rate same-as-last-hour is at least as good as Model C in events found.

What has been done so far

Area	What was done	Where it lives
Foundations	Time split, complete hourly table per building, history looked up by time, per-building cut-offs saved before the test, month dropped	`Electricity_Anomaly_Detection_Midterm.ipynb` , master settings cell
Cleaning	Zero runs of 24 or more hours treated as missing. Site 0 converted from kBTU to kWh (x 0.2931). No imputation of readings. Checks for duplicates, negatives and gaps	Data checks section of the notebook, `table_data_checks.csv`
Labels	LEAD 1.0 labels joined to GEPIII readings and grouped into events by length	Part A, `lead_events.csv` , `lead_labels.csv.gz`

Area	What was done	Where it lives
Predictors	Six methods: same as last hour, typical value, models A, B and C, fixed percentile	Part B, `predictions.csv.gz`, `predictor_settings.json`
Scores and cut-offs	Per-building scores, one cut-off per method at the 0.5% quantile of clean validation scores, frozen before the test	Part C, `cutoffs.json`, `cut_building_values.csv`, `cut_fixed_thresholds.csv`, `flags.csv.gz`
Evaluation	Events found, flagged runs, precision, recall by event length, Education-only view	Part D, `eval_summary_test.csv`, `eval_by_length_test.csv`
Equal-rate comparison	All methods at 0.1, 0.25, 0.5, 1, 2 and 5% of hours flagged, per building and pooled	Part E, `eval_equal_rate_*.csv`
Write-up	Midterm report (Docs and LaTeX), slides, script with Q&A notes, plain-English overview, action points, revised plan	Docs, `latex_report/`, `Presentation_Script_with_QA.md`, `ACTION_POINTS.md`, `PROJECT_EXPLAINED.md`

Status of the midterm submission. Report, slides and presentation are done. One check from the action list was still open the last time we looked: comparing every number in the report and slides with the result files. Do it before the final report reuses those numbers.

Data and features used so far

So far we use GEPIII for readings, weather and building facts, and LEAD 1.0 for labels. CU-BEMS has not been used yet. The lists below come from the notebook code, not from memory.

Source	What it is	What we use	What we do not use
GEPIII readings (`train.csv`)	Hourly meter readings for 2016	Electricity only (meter 0), for the 200 labeled buildings, 8,784 hours each. Site 0 converted to kWh.	Other meters (chilled water, steam, hot water). The 2017 test file has no readings.
GEPIII building facts (`building_metadata.csv`)	Facts per building	Site, primary use, floor area (`square_feet`), year built, floor count. Year built is blank for about 66% of rows and floor count for about 76%. Both are kept, because the models accept blanks; the code drops a feature only if it is empty for every training row, which did not happen.	Nothing else is in the file.
GEPIII weather (`weather_train.csv`)	Hourly weather per site	Air temperature, dew temperature, cloud coverage, precipitation in the last hour, sea level pressure, wind speed. Missing values are filled forward in time only.	Wind direction.

Source	What it is	What we use	What we do not use
LEAD 1.0 (`train_features.csv`, the labeled file)	Hour-level anomaly labels made by people from plots, for 200 buildings in 2016. Derived from GEPIII.	Labels only after the predictions are made: to build events, and to score flags. Labels are never given to a model.	Nothing is used for training.

The features the models see.

Group	Features	Notes
Calendar	Hour of day, day of week, weekend flag	The month is dropped on purpose because the test months never appear in training.
Building	Building identity (as a category), site, primary use, floor area, year built, floor count	Building identity lets a model learn each building's own level.
Weather	The six weather variables above	Filled forward in time only, never backward.
History	Previous hour (`lag_1`), same hour yesterday (`lag_24`), same hour last week (`lag_168`)	On the log scale, log(1 + kWh). Looked up by time on the complete hourly grid and verified against a timestamp lookup.

The feature sets. Model A uses calendar, building and weather. Model B adds `lag_24` and `lag_168`. Model C adds `lag_1` as well. All three use the same settings and the same training rows (rows where all three history values exist), so only the information differs.

The other predictors.

- **Same as last hour:** the prediction for an hour is the reading one hour earlier.
- **Typical value:** the training median for that building, hour and weekend flag. It falls back to the building and hour, then the building alone, when a group is empty.
- **Fixed percentile:** no prediction. It flags use above the building's own 99.5th percentile of clean training use.

What the proposal planned but public data cannot give. Class schedules, exam calendars, holiday flags per site and occupancy. GEPIII has no such files, so these features were not used. They stay in the proposal as future work, which is why the holiday filter in the midterm report's remaining work needs a decision (see Decisions).

Two corrections made to earlier documents (2 Nov 2026). The midterm report (section IV-B), the LaTeX report and the plain-English overview list only three weather variables, and list building features as floor area, primary use and site. The notebook uses six weather variables, and also year built, floor count and building identity. The Docs versions of the midterm report, the LaTeX report and the plain-English overview have now been fixed; keep the corrected wording in the final report. Also state one limit: the models are tested on a later period, not on buildings they have never seen, because building identity is a feature.

How the pipeline works as built

This is the exact recipe the midterm numbers came from. Every later step reuses it unchanged, so that new numbers stay comparable with the midterm.

1. Split by time. One cut in time for every building and every model. Train: before 13 Sep 2016. Validation: 13 Sep to 6 Nov 2016. Test: 7 Nov to 31 Dec 2016. Every building has a complete grid of 8,784 hours. Missing hours stay empty and are never invented.

2. Clean. A run of 24 or more exact zeros is treated as missing. Site 0 is converted from kBTU to kWh. About 4.0% of readings are exact zeros. There were no duplicates, negatives or missing values in the raw checks. About 0.074% of rows are not exactly one hour apart, which is why history is built by time and not by row.

3. Predict expected use. Models A, B and C are one histogram gradient boosting regressor (many small decision trees, each correcting the one before) trained on `log(1 + kWh)`. The settings are chosen on validation and are the same for all three: 600 rounds, learning rate 0.05, at most 127 leaves, at least 100 rows per leaf, L2 penalty 1.0 (the notebook flag `TUNE` is off, so these defaults are used). Predictions are converted back to kWh. A model's prediction is left empty where its own history is missing, so all predictors are compared on the same hours.

4. Score each hour. The error is `e = log(1 + actual) - log(1 + predicted)`. For each building and predictor, take the median `m` and the scaled median absolute deviation `s` of the error over clean validation hours (scaled by 1.4826, with a floor of 0.05). The score is `|e - m| / s`. Four buildings with fewer than 500 clean validation hours borrow `m` and `s` from similar buildings.

5. Set one cut-off per predictor. The cut-off is the score that 0.5% of clean validation hours exceed. Hours labeled anomalous are left out of this step only. The cut-offs are saved to `cutoffs.json`, `cut_building_values.csv` and `cut_fixed_thresholds.csv` before the test is scored. The values are then frozen.

6. Flag. An hour is flagged when its score is above the frozen cut-off. The fixed percentile method flags use above the building's own 99.5th percentile of clean training use.

7. Check against events.

- An event is a run of consecutive labeled hours in one building.
- An event is found if at least one of its hours is flagged.
- A run is consecutive flagged hours. Run precision is the share of runs that overlap an event.
- Recall is reported by event length: 1 hour, 2 to 3 hours, 4 to 12 hours, over 12 hours. There is also an Education-only view.
- Hours without a label inside a labeled building count as normal. So some flags counted as wrong may be real unusual hours that the labelers missed.

8. Equal alarm rates (Part E). Every method flags the same share of hours, 0.1, 0.25, 0.5, 1, 2 or 5%. Two modes: each building sets its own threshold, or all buildings share one pooled threshold. Thresholds come from scores only, never from labels.

What is frozen for the final project. The split dates, the cleaning rules, the features, the model settings, the score formula and the event definitions. Any change must be reported as a new design, not as a fix.

What the midterm found, and its limits

The previous hour is the most useful clue, and a simple copy-the-last-hour rule nearly ties the full model. All results below are on the test period (7 Nov to 31 Dec 2016), which has 587 usable labeled events (212 of 1 hour, 92 of 2 to 3 hours, 185 of 4 to 12 hours, 98 over 12 hours).

Method	Events found at 1% flagged, per building	Events found at 1% flagged, shared threshold	Events found at the pre-set cut-offs	R squared	Log error
Same as last hour	38.7%	37.6%	29.1%	0.993	0.200
Model C	35.1%	33.2%	29.0%	0.993	0.227
Typical value	20.8%	4.8%	1.4%	0.958	0.699
Model B	18.2%	10.2%	3.4%	0.978	0.441

Method	Events found at 1% flagged, per building	Events found at 1% flagged, shared threshold	Events found at the pre-set cut-offs	R squared	Log error
Model A	17.9%	5.6%	1.5%	0.955	0.705
Fixed percentile	8.7%	1.7%	7.2%	not applicable	not applicable

By event length. For events over 12 hours, model C finds 50.0% and the last-hour rule 32.7%. For 1-hour and 2-to-3-hour events, the last-hour rule finds 40.6% and 52.2%, and model C finds 25.5% and 45.7%.

Run precision. At 1% flagged, 9 to 18% of flagged runs overlap an event for the five methods that predict. Most flags do not match a labeled event.

What these results mean.

- The previous hour is the main ingredient. Models A and B find about the same share although B predicts better. Yesterday and last week improved prediction but not detection. We have not tested why.
- Predicting well is not detecting well. The last-hour rule has the best accuracy and still finds fewer long events than model C.
- The professor's question has a mixed answer so far. The concern holds for the last-hour rule on long events and is smaller for model C. This was tested directly on 2 November (notebook 09); see the persistence section of the 2 November update.
- A shared threshold does much worse than per-building thresholds for the methods without the previous hour. The reason is not fully explained; see the cut-off rule check in the 2 November update.

The alarm-rate problem. At the pre-set cut-offs, the share of hours flagged on the test period ranged from 0.075% to 1.87% against a 0.5% target. We found this after looking at the test results, kept the cut-offs frozen, reported it, and added the equal-rate comparison.

Limits.

- Labels are human judgments from plots, and LEAD derives from GEPIII, so they are not independent. We measure agreement with people, not truth.
- 200 labeled buildings, one year, one split.
- The midterm confidence intervals treated events as independent. Events in one building are related, so they were too narrow. This was fixed on 2 November: the new intervals resample whole buildings (notebook 08).
- Small groups, for example 36 events in the Education-only view, make differences of a few points noise.

- "Found" needs only one flagged hour. It is the most generous definition.
- Models are tested on a later period for the same buildings, not on new buildings.
- Public data has no occupancy, schedules or maintenance records, so we cannot name causes.

The figures in this table are copied from the midterm report draft. Compare them with the result files before quoting them in the final report.

Update of 2 November 2026: what was added, run and found since the midterm

This section is the newest part of the document. Where it disagrees with an older section, this section is right. The older sections stay because they explain the design.

Full check from the very beginning. We ran every notebook from a clean folder in this order: 01 (LEAD labels and events), 02 (GEPIII data checks), 03 (predictors), 04 (cut-offs and flags), 05 (evaluation against events), 06 (equal alarm rates), 08 (Isolation Forest and intervals) and 07 (summary check). Every notebook finished without error. All outputs of this clean run are *identical* to the outputs of the earlier rerun, down to the last decimal (data checks, predictor accuracy, cut-offs, both evaluation tables, the intervals). So the pipeline is deterministic: running it again gives the same numbers. Against the frozen midterm files, everything that depends only on data matches exactly (data checks, usable events, the same-as-last-hour and typical-value cut-offs). The numbers that depend on the trained models A, B and C are close but not identical (for example the Model C cut-off is 9.45 now against 9.64 at midterm), because a tree model trained again with another library version is slightly different. The ranking of the six methods did not change. Notebook 07 prints this check.

Isolation Forest (new method, notebook 08, part A). An Isolation Forest is a detector that needs no prediction. It builds many random decision trees and counts how quickly an hour can be separated from the rest; hours that are separated quickly are unusual. We fit one forest per building, on that building's training hours only (before 13 September 2016), with 100 trees and 256 hours per tree, random seed 0, and no labels. Two variants were tested. Variant 1 looks at the Model C error, the hour of day and the weekend flag. Variant 2 looks at the log reading, the change from the previous hour, the change from 24 hours earlier, the hour and the weekend flag. The anomaly score is the negative of scikit-learn's score_samples value, so higher means more unusual. The cut-off follows the same rule as the other methods: the score exceeded by 0.5% of normal validation hours, pooled over all buildings (0.6907 for variant 1 and 0.6920 for variant 2), saved to results/cutoffs_iforest.json. A reader should know one limit: the forest is trained on hours that contain unlabeled and labeled anomalies alike, which is normal for this kind of detector.

Equal alarm rates with margins of error (notebook 08, part B). For each method we flagged the same share of hours, 0.5%, 1% or 2%, in two modes: *pooled* (one threshold for all buildings) and *per building* (each building flags its own top hours). We then counted how many of the 587 usable test events contained at least one flagged hour. The 95% intervals come from a bootstrap that resamples whole buildings 2,000 times (seed 0), because hours and events in one building are not independent. The table gives the result at 1% of hours flagged, per building (all values are percent of the 587 events):

Method	Events found	Recall	95% interval	Median delay (h)	Share of an event's hours flagged
Same as last hour	227	38.7	32.8 to 44.5	1	24.2
Model C	208	35.4	30.0 to 41.1	1	17.8
Isolation Forest, raw readings	175	29.8	23.9 to 36.5	1	13.8
Isolation Forest, Model C error	168	28.6	23.7 to 34.1	1	13.7
Historical typical value	122	20.8	15.7 to 26.3	1	8.2
Model B	106	18.1	13.8 to 22.8	1	7.2
Model A	105	17.9	13.7 to 22.6	1	6.9
Fixed percentile	51	8.7	5.1 to 13.3	1	4.5

The pooled mode gives the same order (last hour 37.6, Model C 34.1, Isolation Forest 28.3 and 23.3, then 10.4, 5.6, 4.8 and 1.7). The full tables are in results/eval_equal_rate_ci_ext.csv. The "share of an event's hours flagged" column is step 2.1 of the plan, now repeated at equal rates. Among events that were found at all, a method flags about half to three quarters of the hours inside them (for Model C, 50% per building and 62% pooled).

What the intervals say. Model C and same-as-last-hour are statistically tied at equal alarm rate. The difference, Model C minus last hour, is -3.2 points at 1% per building with a 95% interval of -7.7 to +1.0, and +0.5 points at 2% (interval -3.4 to +4.7). The interval includes zero, so we cannot say either one is better. Model C is clearly better than the typical-value baseline (+14.7 points, interval 9.4 to 19.5) and than Models A and B. The Isolation Forest is clearly worse than same-as-last-hour (about -9 to -10 points at 1%, intervals that exclude zero) and worse than Model C. Details: results/eval_equal_rate_diff_ext.csv.

What this means for the professor's question. Better prediction from history does not make the tool miss persistent anomalies in these data, but the best predictor is the one that uses the previous hour, and a plain copy of the previous hour does as well. The honest

message for the report: history is what matters, and a complicated model adds precision (Model C's precision at the top of the ranked list is higher than last hour's: 72% against 54% at the top 50, midterm cut-offs) but not more events found.

Leaving labeled hours out of training (task T40). In the design used until now, hours that LEAD marks as anomalous were left out of the scaling and cut-offs but were still in the training data of Models A, B and C and of the typical-value baseline. We retrained with those hours removed (notebook label_excluded/03_gepiii_predictors_label_excluded.ipynb, then notebooks 04 to 06, all other settings unchanged). The effect on the test is small and in the helpful direction: Model C finds 38.0% of events at 1% flagged (both modes), against 35.4% per building and 34.1% pooled before; Model A 19.6%, Model B 19.4% per building. The same-as-last-hour numbers are unchanged because it has no trained part. This is a single training run, so a difference of a few points should not be over-read. Test RMSLE of Model C is 0.200 against 0.200 for last hour, so prediction accuracy is unchanged. The label-excluded results are in results/label_excluded/. The main results in this document still use the original design; the report should say that excluding the labeled hours gives similar or slightly better results.

Choosing the cut-off rule on validation (task T25). Question: would giving each building its own cut-off, taken from that building's clean validation scores, do better than one pooled cut-off? On validation events, ranking within each building beats the pooled threshold for almost every method (for Model C, 44.8% against 41.0% at 1% flagged). But a frozen per-building cut-off does not keep the alarm rate under control on test: instead of 0.5% of hours it flags 1.2% (last hour) up to 7.5% (typical value), because the November to December test period behaves differently from the validation weeks (results/eval_frozen_rules_test.csv). Its higher recall is paid for with many more alarms, so it is not a fair improvement. Decision: keep the single pooled cut-off as the frozen rule, and report per-building ranking only in the equal-alarm-rate comparison, where the alarm budget is the same for everyone.

The persistence test (task T24, notebook 09). The professor asked whether a model that predicts better from history starts to overlook anomalies that last long. We tested this at 1% of hours flagged, per building, on the 587 usable test events, split by event length (95% intervals from the building bootstrap; files results/persistence_*.csv and persistence_test.png).

Events found (%)	1 h	2-3 h	4-12 h	over 12 h
Same as last hour	40.6	52.2	33.0	32.7
Model C	25.9	47.8	32.4	50.0

Events found (%)	1 h	2-3 h	4-12 h	over 12 h
Isolation Forest (raw)	15.1	45.7	29.2	48.0
Historical typical value	3.3	28.3	25.4	42.9

For events over 12 hours, Model C finds 17.3 points more than same-as-last-hour (95% interval 7.3 to 29.6); for 1-hour events it finds 14.6 points fewer (interval -21.3 to -8.9). So the plain copy of the previous hour, which is the extreme case of relying on history, is the method that loses long events: after the first hour the previous hour already contains the anomaly. Model C does not lose them, because it also uses yesterday, last week, the calendar and the weather. Model C is not clearly different from the typical-value baseline on events over 12 hours (+7.1 points, interval -7.2 to 19.5), but is clearly better on shorter events.

A second look, inside events longer than 12 hours: the share of hours flagged falls as the event goes on for every method measured. Model C flags 13.5% of hours in the first hour, 14.4% in hours 2-3, 6.1% in hours 4-12 and 4.6% from hour 13 on. Same as last hour flags 3.1%, 6.7%, 0.5% and 2.8%. The typical-value baseline, which uses no recent history at all, also falls (15.1% in hours 4-12 to 7.6% after hour 12). So the fading cannot be blamed on history alone; it may also reflect that the deviation inside a long labeled event is not constant. We do not claim to know the cause. The honest answer for the report: better use of recent history does not make the tool overlook long events compared with the simpler methods, but copying the previous hour does, and flags inside a long event become less frequent as the event continues for all methods.

Flags that match no label (task T29, notebook 10; first half done, hand judgement still needed). Why: the labels were drawn by people looking at graphs, so a flag with no label is not automatically a false alarm; it may be a real unusual hour nobody marked. At the frozen Model C cut-off on the test period there are 911 flagged runs (a run is consecutive flagged hours). Of these, 197 touch a labeled hour and 714 do not. Of the 714 unmatched runs: 588 are a single flagged hour (median length 1 hour, longest 24), 71 are readings of exactly zero, 59 fall in hours when at least 3 buildings and 30% of the buildings of the same site were flagged together (a site-wide event, for example site 14 on 1 December 2016, 07:00 to 13:00, where several buildings drop to zero together), 74 are within 24 hours of a labeled event (probably the same event), and 510 are none of these. The plots show that the top-ranked unmatched runs include real-looking drops to zero for several hours. These are shorter than the 24-hour zero-run rule, so they are treated as real readings, not as missing data. They are probably meter or power outages. They are unusual, but they are not waste. A review pack of 30 runs (15 highest-scoring unmatched and 15 random unmatched), each drawn with three days either side, is in results/unmatched_flags_review.pdf with a sheet to fill in

(results/unmatched_flags_review_sheet.csv). Claude rated all 30 from the plots alone on 2 November (real unusual, plausible, false alarm, cannot tell; the sheet has a note for each). Rules used: real unusual means a clear visible departure a person would plausibly mark (a drop to zero, a spike, a sustained shift); plausible means a modest visible departure; false alarm means nothing beyond ordinary variation, or a timing slip of the normal on/off pattern; cannot tell means the plot does not decide it. Result. All 15 top-ranked unmatched runs are real unusual: they are drops to zero, 8 of them the site-14 outage of 1 December and several at the edge of an event the labelers did mark. Of the 15 random unmatched runs, 3 are real unusual, 4 plausible, 6 false alarms and 2 cannot tell (two flat meters reading 0.04). So the random sample suggests about one in five unmatched flags is a clearly real unusual hour, about a quarter are plausible, and about 40% are false alarms. This is a sample of 15, rated by one rater (Claude) without knowing the buildings, so it is only a rough estimate; the team should check the sheet and change any rating they disagree with. What it tells us: unmatched flags are not all mistakes, a good share are real outages the labelers did not mark, so the precision against labels understates the tool, and zero-reading outages shorter than 24 hours are the most reliable flags we produce.

What this changes in the plan. Done: Isolation Forest (Step 1.2), margins of error (Step 2.2, grouped by building), hours inside events at equal rates (Step 2.1), label-excluded training, cut-off rule check (Step 1.1). Also done: the persistence test (Step 1.3, task T24). Not done: flags that match no label (Step 2.3), more history clues (Step 2.4), the ranked review list (Step 3.1), CU-BEMS, the site-wide explain-away filter, and all writing.

Where the new files are. Notebook 08_isolation_forest_and_intervals.ipynb (run after 06) writes flags_ext.csv.gz, cutoffs_iforest.json, eval_equal_rate_ci_ext.csv, eval_equal_rate_diff_ext.csv and eval_frozen_rules_test.csv. Copies of the small result files are in the project folder results/ (results/label_excluded/ for the retraining). Large files (flags, predictions) stay in outputs_split/. Notebook 09_persistence_test.ipynb writes persistence_recall_by_length.csv, persistence_differences.csv, persistence_flag_rate_by_position.csv and persistence_test.png. Notebook 10_unmatched_flags.ipynb writes flagged_runs_model_C_test.csv, unmatched_runs_tagged.csv, unmatched_flags_review.pdf and unmatched_flags_review_sheet.csv (the 30 sampled runs with the judgement column, filled in by Claude). requirements.txt sits in the project folder.

Update of 9 November 2026: CU-BEMS prepared and tested (tasks T31 and T32)

This section is the newest part of the document. It records what was done with the third dataset, CU-BEMS, after you downloaded it. The plan for CU-BEMS is further down ("How CU-BEMS will be used"); where this section differs from that plan, this section is right and the differences are listed below.

Why CU-BEMS is used this way, and is it a pure test set? (question raised on 9 November). CU-BEMS is used as an evaluation dataset, not as a pure test set. The reason is that every method that uses a model needs the routine of the building it is scoring, so the models (A-prime, B-prime, C-prime and the Isolation Forests) are trained on CU-BEMS's own earlier months (1 July 2018 to 30 June 2019). Problems are planted only in the last three months (October to December 2019), which no model was trained on and nothing was tuned on. Training on GEPIII and applying the result to CU-BEMS was not chosen because CU-BEMS has no weather and no building facts, so the features differ. If a pure evaluation-only version is wanted, the only methods that could be used are the two that need no fitting: same as last hour and the typical value. What it adds to the project: it is the required third dataset; it measures what labels cannot (smallest rise a method can find, effect of length, the professor's persistence question with known start and length of every problem); and it is a second building type in a tropical climate. The user (Barnnita) believes the downloaded data was complete when downloaded; the floor 6 file for 2019 nevertheless ends in the middle of a row on 22 October 2019, so it may have been cut off at the source. The figshare file size should be compared; if the original is complete, redownload and rerun notebooks 11 and 12 (about 20 minutes).

What we received (T31). Fourteen CSV files in the project folder data/cu_bems/: 2018Floor1 to 2018Floor7 (1 July to 31 December 2018, 264,960 one-minute rows each) and 2019Floor1 to 2019Floor7 (2019, 525,600 rows each, except Floor 6, see next paragraph). Each file has one row per minute and, for each zone, the power of every air-conditioning unit, light circuit and plug circuit in kW, plus temperature, humidity and light sensors. We use only the power columns (kW). The folder also holds two PDF files (a device catalogue and a Sentron meter manual) that we do not use. The number of power meters per floor is 11 (floor 1), 24 (floor 2) and 17 (floors 3 to 7).

A problem to check: the 2019 file for floor 6 is short. 2019Floor6.csv stops on 22 October 2019 at 13:27 (424,168 usable rows), and its last line is a stray "2" that is not a date. We dropped that line. Because of this, floor 6 has no data from 22 October to 31 December 2019, which is inside our test period. We do not know whether the original file is also short or whether the download was cut off. Barnnita should compare the file with the figshare page. The other thirteen files are complete in length.

Hourly series (notebook 11_cubems_hourly_series.ipynb). For each meter we average the kW over the minutes of each hour (which equals kWh for that hour). A meter counts for an hour only if it has at least 45 of the 60 minutes. A floor-hour is valid only if every meter on that floor counts; otherwise the hour is left missing and nothing is filled in. A floor's value is the sum of its meters. We make eight series: the seven floors and a "six-floor total" (floors 1 to 5 and 7), because floor 6 is missing at the end. This replaces the plan's "building total plus 7 floors". The share of valid hours over the whole period is 96.1% (floor 1), 96.1% (floor 2), 86.0% (floor 3), 76.2% (floor 4), 92.6% (floor 5), 86.9% (floor 6), 95.9% (floor 7) and 65.1% (six-floor total). The gaps are mostly floors 3 and 4 in October and November 2018, floor 4 from March to May 2019, and floors 2, 3, 5 and 7 in January and February 2019. There are no negative values and no exact zeros in any floor series, so the 24-hour zero rule used for GEPIII changes nothing here. The typical hourly use (median kWh) is about 103 for floor 1, 8.5 for floor 2, 1.1 for floor 3, 3.4 for floor 4, 1.6 for floor 5, 5.1 for floor 6 and 3.6 for floor 7, and about 164 for the six-floor total. We did not look for the reason floor 1 is so much larger than the others.

Split. Training 1 July 2018 to 30 June 2019, validation 1 July to 30 September 2019, test 1 October to 31 December 2019 (as proposed in the plan). Valid hours in validation: 99.6% for every series. In test: nearly 100% for floors 1, 2, 3, 5, 7, 96.8% for floor 4 and the six-floor total, and only 23.4% for floor 6.

Planted-anomaly test (notebook 12_cubems_planted_test.ipynb).

- Models: A-prime (hour, weekday, weekend flag, series number), B-prime (adds the same hour yesterday and last week) and C-prime (adds the previous hour). They use the midterm settings (HistGradientBoostingRegressor on log1p of the reading, 600 rounds, learning rate 0.05, 127 leaves, minimum 100 rows per leaf, l2 1.0, seed 0), trained once on the training period. Baselines: same as last hour and the typical value for that series, hour and weekend flag. One Isolation Forest per series (variant 2 features: log reading, change from the previous hour, change from 24 hours earlier, hour, weekend flag), fitted on training hours, no labels.
- Scores: as in the midterm, the gap on the log scale, divided by the series' own usual gap (median and scaled median absolute deviation from the validation period, with a floor of 0.05).
- Planting: in the test period only. For each series, 16 problems: each of the lengths 1, 3, 8 and 24 hours, each with four shapes (use multiplied by 1.3, 1.6 or 2.0, or a drop to 5% of the reading). Problems do not overlap, are at least 48 hours apart, and lie only on hours with a real reading. About 119 problems and about 1,126 problem-hours per placement (about 7% of valid test hours). All history features and predictions are recomputed after planting, because a real problem also enters the history. The whole procedure was repeated for 20 random placements (seeds 1000 to 1019), and we report the mean over the 20.

- Flags: at equal alarm rates of 0.5%, 1%, 2% and 5% of test hours in each series, and also with the midterm-style frozen cut-off (0.5% of clean validation scores). A problem is found if at least one of its hours is flagged.
- Differences from the plan, all chosen to keep the test clean or the design balanced: (1) problems were planted only in test, not in validation, because nothing is tuned on planted data; (2) 16 problems per series with every shape at every length, which gives about 7% of hours instead of "about 5% or less"; (3) the cut-off rule from Step 1.1 was not used because Step 1.1 kept the single shared cut-off; (4) the scaling comes from the unplanted validation period, whose scores are computed from unplanted history; (5) the equal-rate level 5% was added.

Result 1: share of planted problems found (mean over 20 placements, percent).

Method	1% flagged	2% flagged	5% flagged	Frozen cut-off (flag rate)
Same as last hour	25	32	44	15 (0.52%)
Historical typical value	9	18	34	3 (0.94%)
Model A-prime	10	18	35	5 (1.03%)
Model B-prime	12	18	33	4 (0.80%)
Model C-prime	27	41	65	24 (0.94%)
Isolation Forest	23	33	47	18 (0.61%)

Model C-prime finds the most. Against the last-hour rule, its gain is +2.2 points at 1% (but between -5.1 and +8.4 across the 20 placements, so not a clear difference), +9.1 points at 2% (between +2.5 and +16.0), +21.2 points at 5% and +8.6 points with the frozen cut-off. Against the Isolation Forest the gain is +3.5 points at 1% (between -4.2 and +9.2) and +8.4 at 2% (between -0.8 and +15.1). Even the best method finds only 41 in 100 problems at 2% of hours flagged, so the numbers are modest in absolute terms.

Result 2: by size and by length at 2% flagged (percent found).

Method	Drop	x1.3	x1.6	x2.0	1 h	3 h	8 h	24 h
Same as last hour	59	19	24	28	20	25	31	50
Historical typical value	55	4	5	8	12	13	17	29
Model A-prime	57	3	5	9	12	13	17	29
Model B-prime	62	3	3	6	11	13	17	30

Method	Drop	x1.3	x1.6	x2.0	1 h	3 h	8 h	24 h
Model C-prime	86	10	22	50	30	34	41	59
Isolation Forest	63	15	19	36	25	29	31	46

A drop to near zero is easy for every method. A 30% rise is hard for every method (at most 19%). Model C-prime is clearly best for a doubling (50%). Longer problems are found more often simply because they give more hours to flag.

Result 3: the professor's question on planted 24-hour problems (figure cubems_persistence_24h.png). For the 24-hour problems we measured how often each hour of the problem is flagged, at 1% of hours flagged. Average flag rate in the first 6 hours, the middle hours and the last 6 hours: last-hour rule 3.5%, 0.9%, 1.0%; Model C-prime 6.3%, 4.1%, 4.5%; Isolation Forest 4.7%, 2.1%, 2.2%; model A-prime 5.5%, 6.1%, 5.9%; model B-prime 5.2%, 5.6%, 6.0%; typical value 5.3%, 5.8%, 5.5%. So the last-hour rule loses almost all grip after the first hours, which is the worry the professor raised. Model C-prime also drops after the start (by roughly a third) but stays well above the last-hour rule. The methods that do not use the previous hour (A-prime, B-prime, typical value) stay level, but they are weak at finding small rises. This agrees with the LEAD result (notebook 09) and gives a second view of it. It holds for a flat multiplier planted by us; real long faults may look different.

Limits to state next to every CU-BEMS number. One building, and its floors are related to each other, so the 20 placements are not 20 independent buildings. The problems are simple and our own design, so this is not real-world performance. There are no human labels, so this says nothing about agreement with people. Only recall is measured; flags outside planted events are not necessarily false alarms. Thai public holidays are not modelled, so some flags outside the planted problems may be holidays (not checked). Floor 6 is missing from 22 October 2019. There is no weather. Nothing was tuned on CU-BEMS.

Where the new files are. Notebooks 11_cubems_hourly_series.ipynb and 12_cubems_planted_test.ipynb in notebooks/. Data: data/cu_bems_hourly/cu_bems_hourly_wide.csv.gz (the eight hourly series), device_stats.csv (missing and zero share per meter). Results in results/cu_bems/: cubems_recall_tables.csv (found share by method, rate, length and shape, with spread over the 20 placements), cubems_event_results.csv.gz (every planted problem in every placement), cubems_planted_events.csv, cubems_persistence_24h.csv and cubems_persistence_24h.png, cubems_persistence_24h_rate.csv, cubems_cutoffs.json.

Everything that remains, from 2 to 17 November 2026 (supersedes the older table below)

1. **Confirm test protection (option A)** with the team. Five minutes.
2. **Persistent-anomaly test (Step 1.3), DONE 2 Nov.** Use the existing results to answer the professor's question directly: for events of 1 hour, 2 to 3 hours, 4 to 12 hours and over 12 hours, show whether Model C loses ground against the typical-value baseline as events get longer. The by-length tables already exist (eval_equal_rate_by_length_test.csv); this is mostly analysis and a figure.
3. **Flags that match no label (Step 2.3), DONE on 2 November (notebook 10; Claude filled in the ratings, so the team should cross-check them).** Sample and look at flagged runs with no label to see whether they are plausible anomalies LEAD missed or false alarms; report the share.
4. **More history clues (Step 2.4).** Optional: lags of 2 and 3 hours and a 24-hour average. Drop first if time is short.
5. **CU-BEMS dataset** (T31 and T32), DONE on 9 November: downloaded, aggregated to hourly, own time split, anomalies planted in the test period, all methods run, results by length and size. This gives the required third dataset. See the update of 9 November. Still open: check the short floor 6 file, and write up the limits.
6. **Site-wide explain-away filter.** Mark a flag as explained when most buildings of a site deviate together in the same hour; count how many flags it removes and whether recall falls. About 0.5 day.
7. **Ranked review list (Step 3.1).** Produce the top flagged runs per building with score and date for a person to review. About 0.5 day.
8. **Report writing, in the AIT thesis template:** map the sections to the template; Related works with citations (literature on anomaly detection in building energy data, Isolation Forest, gradient boosting, and the BDG2, LEAD and CU-BEMS papers; find and verify each source before citing it); Discussions on performance, interpretability (what a flag means, which features matter) and complexity (training time, model size, why a baseline is competitive); Conclusions; honest comparison with other approaches, with the sentence that same-as-last-hour is as good as Model C at equal alarm rate.
9. **Slides and script** for the presentation.
10. **Final tables and figures**, tidy the folder (requirements.txt is already written), delete CP-LAB/_rerun_tmp and any temporary folders.

Suggested timetable: 2 to 5 November, items 2 and 3 and start item 5; 6 to 9 November, items 5, 6, 7; 10 to 14 November, item 8 (writing) and item 9; 15 to 16 November, item 10 and a full read-through; 17 November, submit. If time runs short, drop item 4 first, then the optional extras.

What is left to do

Status on 2 November: this older plan is kept because it explains why each step exists. Steps 1.1, 1.2, 1.3, 2.1, 2.2 and 2.3 are done, Step 0.1 is partly done (requirements.txt and READMEs exist; pushing to a shared repository and the final tidy remain), and Steps 2.4, 3.1, 4.1 and the CU-BEMS check remain. The newest list is Everything that remains, above. Original text: ten steps remain, in five groups (housekeeping, core, evidence, product, write-up), plus CU-BEMS (next section). The core steps total about 10 to 12 working days of effort for two people. CU-BEMS adds about 1.5 to 2 days. These are our estimates and do not come from the professor or the course. The final deadline is not known, so the steps are in order and not tied to dates.

Step	What	Effort	Depends on
0.1	Housekeeping: fix wording, number check, repository, starter notebook, test guard	1 day	Nothing
1.1	Choose cut-offs using labeled events from validation	1 day	0.1
1.2	Add an Isolation Forest detector	1 to 2 days	0.1
1.3	Test persistent anomalies directly	1 day	0.1
2.1	Count hours inside events and delay to the first flag	0.5 to 1 day	0.1
2.2	Margins of error that treat each building as one unit	0.5 to 1 day	1.1, 1.2
2.3	Study flags that match no label	1 day	1.1
2.4	Try more history clues	0.5 day	0.1
3.1	Build the ranked review list	1 day	1.1, 1.2, 1.3
4.1	Final report, slides, script, Q&A, notebook cleanup	2 days	All of the above
C	CU-BEMS check with planted problems (next section)	1.5 to 2 days	1.1, 1.2, 1.3

Rules for every step.

- Each step is its own notebook in the folder `final/`. It loads the frozen midterm outputs read-only and writes only into its own output folder.

- Start each notebook with a reproduction check. Rebuild one midterm number, for example last hour at 1% per building (38.7%), from the loaded files. If it does not match, stop.
- Make every new choice on the validation period only. Score the test period once for each frozen design. Keep a guard flag in the starter file, `TEST_UNLOCKED = False`, and switch it on only for that scoring.
- Compare every new method with the midterm methods at equal alarm rates, per building and shared, on the same hours.
- Any change to the split, cleaning, features, model settings, score formula or event definitions is a new design. Name it and report it as one.

Step 0.1 Housekeeping

- Fix the feature wording in the report, LaTeX report and plain-English overview (see Data and features).
- Compare every number in the report and slides with the result files, and keep the list of checked values.
- Push the midterm notebook and small result files to the shared repository. Ignore the large raw files. Write a README with the run instructions. Move older notebooks into an `archive/` folder.
- Build the starter file: settings, loaders for the frozen midterm outputs, the shared helper functions copied unchanged from Parts C to E, the reproduction check, the test guard and a table-saving helper. Put the helpers in a `common.py` so two people do not edit the same function.
- **Done when** both of us can clone the repository, run the starter, and see the reproduction check pass.

Step 1.1 Cut-offs chosen from labeled events

- **Why.** The midterm cut-offs are set from normal validation hours only (labeled hours are left out), not tuned on labeled events, and the flag rates they gave on test were uneven.
- **First, count** the labeled events per building in the validation period. We have not counted them. If most buildings have only a few, a per-building rule is noisy.
- **Method.** Compare, on validation only, (a) the current single cut-off per method, (b) a fixed alarm rate per building, and (c) a per-building cut-off chosen to find the most validation events at a fixed alarm budget, with buildings that have few events borrowing from buildings of the same primary use. The borrowing weight is a setting we choose on validation.

- **Freeze** the chosen rule and its values to files. Score the test once.
- **Report** events found, run precision and the share of hours flagged, against the midterm numbers, and say whether the uneven flag rates are fixed.

Step 1.2 A different kind of detector

- **Why.** The research question names the detector. So far we only change the predictor.
- **Method.** Use scikit-learn's Isolation Forest, which isolates hours that are unlike the rest. Fit on training rows only, with no labels. Two variants: (i) on the model C score plus hour and weekend flag, (ii) on the log reading, the changes from the previous hour and from the same hour yesterday, hour and weekend flag. The threshold is set by alarm rate, as for the other methods.
- **Report** the Part E tables with the Isolation Forest added, per building and shared.
- **Expect nothing in advance.** It may do worse. That is a valid finding.

Step 1.3 Persistent anomalies

- **Why.** The professor asked whether better prediction from history hides long-lasting anomalies. So far we only compared two methods by event length.
- **Test A: remove history.** Retrain model C without yesterday and last week, keeping only the previous hour. Compare recall by event length at equal alarm rates. If long events are lost, those inputs explain why C keeps flagging them. This tests our untested explanation directly.
- **Test B: score that remembers.** Replace each hour's score with the mean of the scores over the last 3, 6, 12 or 24 hours. Choose the window on validation. Check whether long events stay flagged and what it costs on short events.
- **Test C: look inside events.** For each found event, record which hours are flagged, as in Step 2.1.
- **Done when** one table by event length and a short written answer exist. CU-BEMS gives a second view, because the true length of every planted event is known.

Step 2.1 Hours inside events and delay (partly done: reported at the pre-set cut-offs in eval_summary_test.csv; the repeat at equal rates remains)

- For each event, compute the share of its hours that are flagged and the number of hours before the first flag. Report by event length at 1% per building.
- **Why.** One flagged hour currently counts as finding an event.

Step 2.2 Margins of error

- Resample buildings with replacement, for example 1,000 times, and recompute events found and run precision. Report 95% intervals.
- **Why.** Events in one building are related, so the current intervals are too narrow.

Step 2.3 Flags that match no label

- Take about 15 flagged runs with no overlapping event, spread across buildings, for model C and for the last-hour rule at 1%. Plot the actual and expected use for the surrounding week.
- Both of us sort them independently into groups, for example a long unexplained rise, a single spike, a data problem such as a sudden step or zeros, or an ordinary change in pattern. Report how often we agree.
- **Say plainly** that this sorting is our own judgment. It shows whether low precision is partly missing labels. It does not prove it.

Step 2.4 More history clues

- Add two and three hours back and a 24-hour mean, on the log scale, as a new feature set D. Compare it with C at equal alarm rates. The midterm showed history beyond the previous hour added little, so treat this as an extension.

Step 3.1 The ranked review list

- **Content.** For each building, a list of flagged runs in order of how unusual they are. For each run: building, start time, length in hours, peak score, actual use, expected use, the gap as a percentage, hour and weekday, and the outdoor temperature.
- **Ranking.** By the run's peak score, within building and across buildings. Report precision at k, as in the midterm.
- **Use.** The final design's best method. A one-page guide explains how to read it.
- **Limit.** The list says where to look. It does not say why.

Step 4.1 Write-up

- Final report in the same format as the midterm, slides, script with Q&A notes, and a clean notebook set that runs top to bottom. Every number is checked against the result files. The limits section lists what was not tested.

How CU-BEMS will be used

CU-BEMS is a second dataset that lets us test our methods where the right answer is known for certain, because we plant the problems ourselves. It cannot tell us how well flags agree with human judgment, because it has no labels. LEAD gives us real but imperfect labels. CU-BEMS gives us exact but artificial ones. The two answer different questions.

What CU-BEMS contains. These facts come from the dataset's paper (Pipattanasomporn et al., Scientific Data, 2020).

Item	Fact
Building	One seven-storey academic office building at Chulalongkorn University, Bangkok, about 11,700 square metres
Period	1 July 2018 to 31 December 2019, 18 months
Resolution	One minute, in kW
What is measured	55 air-conditioning units, 33 lighting loads and 32 plug loads, in 33 zones across 7 floors. Elevators and exit signs are not measured (about 1 to 2% of load).
Indoor sensors	Temperature, humidity and light at 24 locations. Taken down for maintenance for part of the period.
Files	14 CSV files, one per floor per year, 30 to 80 MB each. Data on figshare, DOI 10.6084/m9.figshare.11726517.
Gaps	Most series are at least 95% available. Some air-conditioning units on floors 3 and 4 miss about 10 to 20%.
Not included	Outdoor weather. Anomaly or fault labels. Building facts such as year built.
Licence	The article is CC BY 4.0. The paper does not state a separate licence for the data files, so cite the paper and check the figshare page before publishing anything derived from the data.

What it is good for. A second building type in a tropical climate, closer to the AIT setting than the American and European sites in GEPIII. Fine detail by floor. A clean test of long-lasting anomalies, because we know the true length of every planted event. It is the closest public stand-in for AIT data, which we do not have.

What it cannot do.

- It is one building. Results from it do not show that a method works across many buildings.
- It has no human labels, so it cannot measure agreement with people.
- Planted problems are simple and our own design. A method can find them more easily than real faults, so success here must not be reported as real-world performance.
- The floor and zone series of one building are related to each other, so they are not independent buildings.

The plan.

1. **Get and read the data.** Download the 14 files on a machine with access to figshare. Check column names, units, the timestamp and the gaps.
2. **Build hourly series.** Convert one-minute kW to hourly kWh (the mean kW over the hour, times one hour). Mark an hour as missing when too many of its minutes are missing; pick that rule after looking at the gaps. Use the building total and the 7 floor totals, which is 8 series. Do not use single devices, because they are too noisy to read. Check before applying the 24-hour zero-run rule, because zero use at night can be real here.
3. **Features.** Calendar (hour, day of week, weekend flag), history (previous hour, same hour yesterday, same hour last week, on the log scale) and the series identity. There is no weather and no building facts. So the three models become: A-prime with calendar and series identity, B-prime adding yesterday and last week, C-prime adding the previous hour. Keep the same model settings as the midterm. Weather from an outside source and a Thai holiday calendar are not planned. Add them only if time remains and say so.
4. **Split by time.** Proposal: train 1 July 2018 to 30 June 2019, validation 1 July to 30 September 2019, test 1 October to 31 December 2019. Confirm after checking the gaps on floors 3 and 4.
5. **Plant problems.** Plant them in the hourly series before building the history features, because a real problem also feeds the history. Plant them in validation and in test with different random seeds. Plant none in training.
    - **Lengths:** 1, 3, 8 and 24 hours, matching our event-length groups.
    - **Shapes:** a rise, where the reading is multiplied by 1.3, 1.6 or 2.0, and a drop to near zero. These are our assumptions.

    - **Rules:** no overlap, at least 48 hours apart, only on hours with a real reading, and a small share of hours (about 5% or less). Repeat with about 20 random seeds and report the mean and the spread.
6. **Run the methods.** Same as last hour, typical value, A-prime, B-prime, C-prime, the Isolation Forest and the cut-off rule from Step 1.1. Cut-offs come from clean validation hours with planted hours left out, as in the midterm. Score the test once.
7. **Measure.** Events found at equal alarm rates of 0.5, 1 and 2%. Found by length and by size of rise, which shows the smallest change each method can find. Share of hours inside each event that are flagged, and the delay to the first flag. For the 24-hour events, a plot of how the flagging fades or holds by hour since the start, for last hour, C-prime and the score that remembers. This is the clean answer to the professor's persistence question.
8. **Report.** One table by length and size of rise, one persistence figure, and a short paragraph that states the limits above. Flags outside planted events are not necessarily false alarms, because the real data may hold real anomalies. Report recall as the main result and treat precision as a lower bound.

Effort. About 1.5 to 2 days. Reading and hourly conversion take about half to one day. Planting and running take about half to one day. The rest is writing.

BDG2. The midterm report said we would compare with BDG2. Decision (2 Nov 2026): change that wording to say BDG2 is the source collection of GEPIII, and that our three datasets are GEPIII, LEAD and CU-BEMS. GEPIII is a subset of BDG2, according to the BDG2 paper (Miller et al., 2020), so BDG2 is not independent evidence for the 2016 buildings. BDG2 also has 2017 readings for the same buildings but no labels for them. The comparison is not planned. If time remains, 2017 can be used to check how prediction accuracy holds a year later, and nothing more.

What the finished project looks like

The finished project is a tested method plus a ranked list a person can use, with an honest statement of how far it can be trusted. It does not decide what is wrong.

What we hand in.

- **A set of notebooks** that run from the frozen midterm outputs through each final step, one notebook per step, with a short README. The midterm notebook stays as it is.
- **A shared repository** with the code, the small result files and the instructions. Large raw files are left out.
- **Result tables** for the cut-off rule, the detector comparison, the persistent-anomaly test, hours inside events and delay, margins of error, and the CU-BEMS check.
- **A short study of the flags that match no label**, with worked examples.

- **A ranked list of unusual hours** for each building, with actual use, expected use, size of the gap, length of the run and the outdoor temperature, plus a one-page guide.
- **A final report** in the same format as the midterm report, **slides**, a **script** and **Q&A notes**.

What the final report answers.

1. How well do flags from an expected-use model match the labeled events? The midterm gives the first answer. The final report adds the label-guided cut-off, a second detector and margins of error.
2. Which design choices help: the clues (done), the cut-off (Step 1.1) and the detector (Step 1.2).
3. Does better prediction hide long-lasting anomalies? Step 1.3, with the CU-BEMS check as a second view.
4. What do we still not know? Why the shared threshold does worse, and what the unmatched flags are, unless the steps above settle them.

We do not know in advance what these steps will show. The final report says what they show, including if they show that a simple rule is enough.

What the project will not contain.

- Live alerts, measured savings or staff time saved. This is out of scope and stays as future work.
- AIT meter data, unless access is confirmed. It is not being attempted.
- Claims about waste, faults or savings. A flag means unusual use for its context.
- A holiday filter, unless a calendar is found. See Decisions.

Decisions, risks and sources

Decisions to make now.

1. **The final deadline and what to hand in.** Not known. Everything above is in order and not tied to dates.
2. **The professor's comments after the midterm.** They are not in this document yet. Add them and adjust the steps.
3. **How to protect the final test.** We have already looked at the whole test period. Option A (recommended; assumed so far, still to be confirmed with Arpan): make every new choice on validation only, score the test once per final design, and say in the report that the test was also used for the midterm. This keeps all 587 events. Option B: split the test period in two, decide on the first part and report on the second. It is cleaner but leaves about half the events, so numbers are noisier.

4. **Holiday filter.** The midterm report promises a step that removes flags with a known cause such as a holiday or extreme weather. GEPIII has no holiday calendar per site. Either build only a weather version, or drop the holiday wording from the final report.
5. **BDG2 wording.** Change "compare with BDG2" in the report, as explained in the CU-BEMS section.
6. **CU-BEMS in the final.** Recommended: yes, after the core steps. Cut it first if time runs short, together with Step 2.4.

Risks.

- **Few labeled events per building in validation.** A cut-off chosen from a handful of events is noisy. Count them first (Step 1.1) and borrow from similar buildings.
- **The Isolation Forest may do worse.** Report it plainly.
- **Precision may stay low.** Step 2.3 shows whether missing labels are part of the reason.
- **Planted problems are easy.** CU-BEMS results can look better than real-world performance. State this beside every CU-BEMS number.
- **CU-BEMS data handling.** 14 large files, gaps on floors 3 and 4, and an unclear licence for the data files. Check the figshare page and cite the paper.
- **Numbers drift.** Check every number in the final report against the result files.
- **Time.** The write-up is easy to squeeze. Protect two days for it.

What to cut first if time runs short. CU-BEMS, then Step 2.4, then Step 2.3 down to 8 examples, then Step 2.1 if Step 1.3 already answers the question. Never cut Steps 1.1, 1.2, 1.3, 3.1 and 4.1.

Rules we keep.

- Say "unusual for its context" and "worth a human check". Never waste, fault or savings.
- Say "we have not tested that yet" when we have not, and give the plan.
- One person edits a notebook at a time. Each person reads the other's part of the report before it goes out.
- Short, plain sentences. No jargon without a one-line explanation.

Sources.

- CU-BEMS, smart building electricity consumption and indoor environmental sensor datasets (Pipattanasomporn et al., 2020). Opened and read for this document.
- The Building Data Genome Project 2 (Miller et al., 2020). The abstract was opened for the building counts and the relation to GEPIII.
- GEPIII overview and LEAD 1.0 (Gulati and Arjunan, 2022) are cited in the midterm report. They were not opened again for this document.

- The midterm notebook `Electricity_Anomaly_Detection_Midterm.ipynb`, read for the features, split and settings.
- The midterm report draft, for the result figures. The result files on your machine were not available here to recheck them.